

République du Cameroun Paix-
travail-patrie

Ministère de l'Enseignement
Supérieur

Université de Maroua

Ecole Nationale Supérieure
Polytechnique de Maroua

Département d'Informatique et
Télécommunication



Republic of Cameroon
Peace_work_Fatherland

Ministry of Higher Education

The University of Maroua

National Advanced School of
Engineering of Maroua

Computer Science and
Telecommunication department

YB-TECHs

French AI Startup · yb-techs.eu

RAPPORT DE STAGE

**Agent Intelligent de Collecte et d'Extraction de Données
par Automatisation GUI/Web**

Du 1^{er} Juin au 31 Août 2026

Stagiaire : NDJOKA NGHOKO Jodelle Baruch 22E0480EP

📍 **Siège social** : Caen, France / Yaoundé, Cameroun
✉ **Email** : ybtechs.io@gmail.com
📞 **Contacts** : +237 6 76 65 71 62 / +33 6 59 28 72 21
🌐 **Site web** : <https://yb-techs.eu>

Encadreur de stage : **Yris Brice Wandji Piugie, PhD — Fondateur & CEO, YB-TECHs**

Année académique : **2025-2026**

Table des matières

1. Introduction	4
1.1 Présentation de l'entreprise d'accueil	4
1.2 Contexte et problématique	4
1.3 Objectifs du stage	4
1.4 Périmètre retenu et démarche	5
1.5 Structure du rapport	5
2. État de l'art	6
2.1 Selenium — Automatisation de navigateur (Couche 1)	6
Origine et évolution	6
Fonctionnement technique	6
Limites identifiées et alternatives	6
2.2 OCR — Reconnaissance optique de caractères (Couche 2)	6
Histoire et état de l'art	6
Deux moteurs candidats	7
Prétraitement d'image	7
Métriques d'évaluation	7
2.3 OmniParser v2 — Analyse d'interfaces graphiques (Couche 3)	7
Genèse et publication	7
Architecture interne (version 2)	7
Positionnement et exigences matérielles	7
2.4 LLM locaux et orchestration	8
2.5 Synthèse comparative	8
3. Architecture du système	10
3.1 Vue d'ensemble	10
3.2 Environnement de développement	10
3.3 Couche 1 — Module Selenium	10
3.4 Couche 2 — Module OCR	11
3.5 Couche 3 — Module OmniParser	11
3.6 Orchestrateur LLM	12
Cascade heuristique vs décision LLM : une clarification nécessaire	12
3.7 Nettoyage et stockage	12
3.8 Interface web (au-delà du périmètre initial)	13
3.9 Schéma architectural global	13
4. Implémentation	15
4.1 Semaine 1 — Initialisation et setup	15
4.2 Semaine 2 — Module Selenium et pipeline de collecte aveugle	15
Le pipeline de collecte aveugle	15

4.3 Semaine 3 — Module OCR	16
4.4 Semaine 4 — Module OmniParser et fiabilisation réseau	17
Fiabilisation réseau en cascade	17
4.5 Semaine 5 — Orchestrateur LLM et formulation de réponse	18
De Phi-3 Mini à Llama 3.2 3B : la formulation de réponse, un chantier à part entière	19
4.6 Semaine 6 — Nettoyage et stockage	20
Un bug silencieux découvert en reconstruisant le module de validation	20
4.7 Semaine 7 — Tests, cas d'usage et livrables	21
4.8 Interface web — un développement complémentaire	21
5. Résultats et indicateurs de performance (KPI)	24
5.1 Synthèse des indicateurs par rapport aux cibles du cahier des charges	24
5.2 Résultats obtenus sur les cas d'usage réels	24
5.3 Indicateurs de suivi du développement	25
5.4 Limites de la mesure	25
6. Difficultés rencontrées et méthodes de diagnostic	26
6.1 Difficultés d'environnement (semaines 1 et 4)	26
Incompatibilité de Python 3.14 avec l'écosystème d'IA	26
ImportError flash_attn requis par Florence-2	26
Blocage réseau en cascade (semaine 4)	26
6.2 Un défaut de conception révélé par l'observation, pas par une erreur (semaine 5)	27
6.3 Fiabilité intermittente de Selenium (semaine 7)	28
6.4 Un bug silencieux dans la validation des données (semaine 6)	28
6.5 Synthèse méthodologique	28
7. Conclusion et perspectives	30
7.1 Bilan du stage	30
7.2 Ce qui reste ouvert	30
7.3 Perspectives à court terme	30
7.4 Apports du stage	31
Bibliographie	33
Publications scientifiques et rapports techniques	33
Documentation officielle des outils et bibliothèques	33
Document interne	33
Annexes	34

1. INTRODUCTION

1.1 Présentation de l'entreprise d'accueil

YB-TECHS est une startup française spécialisée en intelligence artificielle, fondée par **Yris Brice Wandji Piugie** (PhD, ENSICAEN). L'entreprise dispose d'un siège social à Caen, en Normandie, et d'une antenne à Yaoundé, au Cameroun. Elle opère à l'intersection de la recherche appliquée en IA, du conseil et de la transformation numérique pour les petites et moyennes entreprises (PME), avec une orientation particulière vers les marchés francophones d'Afrique.

Trois lignes de services structurent l'activité de YB-TECHS : la collecte et la préparation de données pour des projets d'intelligence artificielle, le conseil en IA et le déploiement de solutions data pour les entreprises, ainsi que la formation et l'accompagnement digital destinés aux marchés francophones africains.

C'est dans le cadre de la première de ces trois lignes de services — la collecte de données — que s'inscrit le projet de stage présenté dans ce rapport.

1.2 Contexte et problématique

Dans le cadre de ses missions clients, YB-TECHS collecte régulièrement des données depuis des sources numériques hétérogènes : des sites web dont le DOM est directement accessible, des documents numérisés ou photographiés (factures, formulaires, rapports), et des interfaces graphiques d'applications propriétaires ne proposant aucun accès programmatique — ni API, ni DOM exploitable.

Ces trois familles de sources exigent aujourd'hui des outils et des compétences radicalement différents : Selenium pour l'automatisation de navigateur, des moteurs OCR pour la lecture d'images, des outils de vision par ordinateur pour l'analyse d'interfaces graphiques natives. Cette fragmentation impose des développements sur mesure pour chaque nouvelle mission, avec un coût en temps et en expertise que YB-TECHS souhaite réduire.

Le problème n'est donc pas un manque d'outils individuels — Selenium, l'OCR et l'analyse d'interfaces graphiques sont chacun des domaines matures — mais l'absence d'une couche d'orchestration capable de choisir automatiquement le bon outil selon la nature de la source, sans que l'utilisateur ait à le préciser lui-même.

1.3 Objectifs du stage

Le stage, d'une durée de deux mois (juillet–août 2026), visait à concevoir, développer et valider un prototype fonctionnel d'un agent intelligent unifiant ces trois approches de collecte au sein d'un pipeline unique, piloté par un modèle de langage (LLM) exécuté localement. Le cahier des charges de YB-TECHS proposait deux niveaux d'ambition :

- **Version A (sans RAG)** : un pipeline d'extraction tri-couche (Selenium / OCR / OmniParser) piloté par un LLM local, constituant le socle obligatoire et le livrable minimum viable (MVP) du stage.
- **Version B (avec RAG)** : une extension de la Version A par deux mécanismes de Retrieval-Augmented Generation — une mémoire des trajectoires d'extraction passées (RAG Point 1) et une interface de requête en langage naturel sur les données collectées (RAG Point 3).

1.4 Périmètre retenu et démarche

Le stage a été mené selon la trajectoire de la Version A : les sept premières semaines ont été consacrées à la construction, à la fiabilisation et à la validation progressive du pipeline tri-couche (Selenium, OCR, OmniParser) et de son orchestrateur LLM, ainsi qu'aux modules de nettoyage, de stockage. La huitième semaine a été réservée à la documentation.

Le volet RAG de la Version B (mémoire des trajectoires et interface de requêtage) n'a pas été engagé dans le temps imparti : la consolidation du pipeline tri-couche et de l'orchestrateur — en particulier la fiabilisation de la formulation de réponse par le LLM — s'est révélée plus longue que prévu, ce qui a mobilisé la majorité du temps disponible jusqu'à la fin de la semaine 7. Ce choix, ses raisons et ses conséquences sont détaillés dans la section Conclusion et perspectives de ce rapport.

Au-delà du périmètre strict du cahier des charges, une interface web (FastAPI) a également été développée en complément du pipeline en ligne de commande / notebook, afin de rendre le système utilisable sans connaissance technique préalable — ce point correspond au livrable « API de service » prévu par le cahier des charges et est présenté en section 3.8 et en section 4.8.

1.5 Structure du rapport

Ce rapport est organisé en sept parties. La section 2 présente l'état de l'art des technologies mobilisées (Selenium, OCR, OmniParser v2, LLM locaux). La section 3 décrit l'architecture technique retenue pour le système final. La section 4 retrace, semaine par semaine, l'implémentation réalisée au cours du stage. La section 5 présente les résultats obtenus et les indicateurs de performance (KPI) mesurés au regard des cibles du cahier des charges. La section 6 revient en détail sur les difficultés techniques rencontrées et les méthodes de diagnostic employées. La section 7, enfin, dresse le bilan du stage et ouvre sur les perspectives à court terme.

2. ÉTAT DE L'ART

La collecte automatisée de données depuis des sources numériques hétérogènes est un enjeu central pour les projets d'intelligence artificielle contemporains. Comme indiqué en introduction, les entreprises comme YB-TECHS doivent agréger des informations depuis trois grandes familles de sources — sites web à DOM accessible, documents numérisés, interfaces graphiques natives sans API — qui relèvent chacune d'un champ technique distinct. Cette section présente, pour chacune de ces trois couches ainsi que pour la couche d'orchestration, l'état de l'art des solutions disponibles, les choix retenus par le cahier des charges de YB-TECHS, et les alternatives sérieuses qui ont été considérées.

2.1 Selenium — Automatisation de navigateur (Couche 1)

Origine et évolution

Selenium est né en 2004 chez ThoughtWorks, à l'initiative de Jason Huggins. Sa composante WebDriver, standardisée par le W3C en 2018 sous le nom de WebDriver Protocol, est aujourd'hui le fondement de l'automatisation de navigateurs sur le web. Selenium 4, sorti en 2021, adopte nativement ce protocole W3C et abandonne le protocole JSON Wire historique, plus ancien et moins interopérable.

Fonctionnement technique

Selenium WebDriver envoie des commandes HTTP à un pilote natif (ChromeDriver pour Google Chrome, GeckoDriver pour Firefox), qui pilote à son tour le navigateur réel au niveau du système d'exploitation. Cette approche permet de rendre le JavaScript exécuté côté client, de gérer les cookies et les sessions, ainsi que les structures complexes telles que les iframes et le Shadow DOM — ce qu'une simple requête HTTP ne permettrait pas.

Limites identifiées et alternatives

La littérature technique identifie plusieurs limites à Selenium : une détection par les mécanismes anti-bot (Cloudflare, DataDome), à laquelle répondent des solutions comme undetected-chromedriver ; une gestion parfois délicate des applications monopage (SPA), qui nécessite des attentes explicites (WebDriverWait) plutôt que des délais fixes ; et surtout, une inapplicabilité totale aux applications desktop natives, dépourvues de DOM. Playwright (Microsoft, 2020) constitue la principale alternative moderne, avec un support asynchrone natif, une exécution plus rapide et un meilleur camouflage anti-détection ; Puppeteer (Google) reste, pour sa part, limité au moteur Chromium. Selenium 4 a été retenu pour ce projet en raison de sa documentation abondante, de son support multilingue et de sa prescription explicite par le cahier des charges de YB-TECHS.

2.2 OCR — Reconnaissance optique de caractères (Couche 2)

Histoire et état de l'art

L'OCR existe comme discipline depuis les années 1950 (travaux de David Shepard). La révolution du deep learning a profondément transformé le domaine : les réseaux CRNN (Convolutional Recurrent Neural Network), apparus vers 2015 et combinant un CNN pour l'extraction de caractéristiques visuelles avec un LSTM pour le traitement séquentiel, atteignent aujourd'hui des performances proches de celles d'un lecteur humain sur des documents imprimés de bonne qualité.

Deux moteurs candidats

- **Tesseract OCR** : moteur open source maintenu par Google, fondé sur une architecture LSTM depuis sa version 4 (2018). Il prend en charge plus de cent langues et se montre très performant sur des documents propres, mais nettement moins robuste sur des images bruitées en l'absence de prétraitement.
- **EasyOCR (JaidedAI, 2020)** : fondé sur PyTorch, il combine un module CRAFT pour la détection de zones de texte et un module CRNN pour la reconnaissance à proprement parler. Nativement multilingue, il se montre plus robuste sur des images imparfaites. C'est le moteur recommandé par le cahier des charges de YB-TECHS.

Prétraitement d'image

La qualité d'un pipeline OCR dépend directement de la qualité du prétraitement appliqué en amont. Les techniques standards mobilisées par ce projet incluent la binarisation adaptative (méthode d'Otsu), le débruitage par filtre gaussien ou médian, la correction de perspective par transformée de Hough, et la dilatation ou l'érosion morphologique, utile pour reconnecter des caractères fragmentés par la numérisation.

Métriques d'évaluation

Deux métriques standards permettent d'évaluer la qualité d'une extraction OCR : le CER (Character Error Rate), qui mesure le ratio de caractères incorrects par rapport à une vérité terrain, et le WER (Word Error Rate), plus sévère, qui compte les mots entiers mal reconnus. Le cahier des charges de YB-TECHS fixe une cible de CER inférieur à 15 % sur image propre.

2.3 OmniParser v2 — Analyse d'interfaces graphiques (Couche 3)

Genèse et publication

OmniParser est publié par Microsoft Research en 2024 (Lu et al., arXiv:2408.00203). Il répond à un besoin central des agents d'intelligence artificielle multimodaux : comprendre n'importe quelle interface graphique à partir de la seule capture d'écran, sans accès au code source ni au DOM sous-jacent.

Architecture interne (version 2)

- **Détecteur YOLO fine-tuné** : entraîné sur un jeu de données propriétaire d'icônes d'interface, il détecte les éléments interactifs (boutons, champs, menus, icônes) et retourne leurs coordonnées sous forme de boîtes englobantes accompagnées d'un score de confiance.
- **Modèle Florence-2 (Microsoft)** : modèle de vision-langage qui génère une description sémantique de chaque région détectée. Il remplace GPT-4V, utilisé dans la première version d'OmniParser, afin de réduire le coût et la latence d'inférence.
- **OCR intégré** : extraction du texte contenu dans les boîtes englobantes détectées — libellés de boutons, contenu des champs de formulaire, titres.

Positionnement et exigences matérielles

OmniParser s'inscrit dans la tendance des « GUI agents », des agents capables d'opérer une interface comme le ferait un utilisateur humain. Il est notamment utilisé comme composant de perception dans des frameworks tels qu'UFO (Microsoft), AppAgent ou ScreenAgent, et constitue actuellement l'outil open source le plus performant sur cette tâche. Son usage reste toutefois exigeant en ressources : un

GPU CUDA est recommandé pour obtenir des temps de traitement raisonnables (moins de 3 secondes par image), le traitement sur CPU seul restant possible mais nettement plus lent (15 à 30 secondes par image) — ce qui a directement conditionné plusieurs choix techniques du projet, détaillés en section 3.

2.4 LLM locaux et orchestration

Le cahier des charges de YB-TECHS préconisait initialement Mistral 7B-Instruct, déployé via Ollama, comme modèle orchestrateur. Compte tenu des contraintes matérielles rencontrées dès la semaine 1 du stage — une machine de travail sans GPU dédié — un modèle alternatif plus léger, Phi-3 Mini (3,8 milliards de paramètres, Microsoft Research, 2024), a été retenu. Ce modèle appartient à la famille Phi, conçue pour maximiser les performances de raisonnement à taille réduite grâce à un jeu de données d'entraînement de haute qualité.

Critère	Phi-3 Mini (3,8 B)	Mistral 7B
RAM requise (quantification Q4)	≈ 3 Go	≈ 5 Go
Vitesse d'inférence sur CPU	Très rapide	Rapide
Qualité de raisonnement	Excellente pour la taille	Excellente
Support du français	Bon	Très bon
Fenêtre de contexte	4K / 128K tokens	32K tokens
Identifiant Ollama	phi3:mini	mistral:7b-instruct

Ce choix, motivé par la légèreté et la rapidité d'inférence sur une machine sans GPU, s'est avéré pertinent pour le déploiement initial et le routage entre les trois couches d'extraction. Il a en revanche révélé ses limites sur une tâche plus exigeante — la formulation d'une réponse en langage naturel fidèle aux données extraites — ce qui a conduit, en semaine 5, à un second changement de modèle vers Llama 3.2 3B, documenté en détail en section 4.5 et section 6.

2.5 Synthèse comparative

Le tableau ci-dessous positionne, pour chaque couche du pipeline, l'outil retenu par rapport à la meilleure alternative identifiée dans la littérature, et rappelle la justification du choix opéré par le cahier des charges de YB-TECHS.

Couche	Outil retenu	Meilleure alternative	Raison du choix
Web / DOM	Selenium 4	Playwright	Documentation, multilingage, prescrit par le CDC
Documents / images	EasyOCR + OpenCV	PaddleOCR	PyTorch natif, multilingue
GUI native	OmniParser v2	UFO (Microsoft)	Open source, autonome
Orchestrateur	Phi-3 Mini / Ollama	Mistral 7B	Légèreté et rapidité sur CPU

L'innovation recherchée par ce projet ne réside donc pas dans un outil pris isolément — chacun de ces quatre composants est un choix éprouvé et documenté — mais dans leur orchestration intelligente au sein d'un pipeline unique, capable de sélectionner automatiquement la couche pertinente selon la

nature de la source à traiter. C'est cette orchestration, ainsi que son architecture technique, qui font l'objet de la section suivante.

3. Architecture du système

3.1 Vue d'ensemble

Le système développé repose sur une architecture en cascade à trois couches d'extraction, orchestrée par un LLM local. Chaque couche couvre un type de source de données spécifique — sites web, documents/images, interfaces graphiques natives — et le LLM joue le rôle de cerveau coordonnateur, chargé d'analyser la requête utilisateur et de décider quelle couche interroger, avant qu'un module de nettoyage normalise les données extraites et qu'un module de stockage/rapport les rende exploitables.

Couche	Outil retenu	Type de source	Cas d'usage
1	Selenium 4	Sites web avec DOM HTML	Portails, marketplaces, intranets, e-commerce
2	EasyOCR + OpenCV	Documents numérisés / images	PDF scannés, formulaires photographiés, captures
3	OmniParser v2	Interfaces GUI natives	Applications Windows sans API, applis mobiles
LLM	Llama 3.2 3B / Ollama	Orchestrateur	Classification, routage, formulation de réponse

Le modèle orchestrateur indiqué dans ce tableau — Llama 3.2 3B — correspond à l'état final du système à l'issue du stage. Comme détaillé en section 2.4 et section 4.5, le modèle initialement déployé en semaine 1 était Phi-3 Mini ; le changement vers Llama 3.2 3B est intervenu en semaine 5, motivé par des limites constatées empiriquement sur la tâche de formulation de réponse.

3.2 Environnement de développement

Contrairement à la recommandation initiale du cahier des charges (Anaconda/Conda), l'environnement de développement a été configuré avec Python venv, qui s'intègre nativement à Visual Studio Code et aux notebooks Jupyter utilisés tout au long du stage. Deux environnements virtuels distincts ont été créés, reflétant l'architecture en deux volets du projet :

- `agent_pipe` — environnement du pipeline principal (Selenium, EasyOCR, OmniParser, PyTorch, PaddlePaddle) ;
- `agent_pipe_rag` — environnement dédié à la couche RAG envisagée pour la Version B (LangChain, ChromaDB, Streamlit), provisionné dès la semaine 1 mais non mobilisé, le stage n'ayant pas engagé le volet RAG (voir section 1.4 et section 7).

Pourquoi deux environnements distincts

Les piles logicielles des deux volets sont incompatibles : OmniParser requiert PyTorch et PaddlePaddle avec des contraintes de version très strictes, tandis que ChromaDB et LangChain ont leurs propres exigences. Un environnement unique aurait généré des conflits de dépendances difficiles à résoudre.

3.3 Couche 1 — Module Selenium

Le module Selenium est structuré en quatre fichiers Python distincts, chacun avec une responsabilité unique (principe de responsabilité unique), afin de faciliter les tests et la maintenance.

Fichier	Rôle
navigator.py	Initialise Chrome avec options anti-détection, timeout, user-agent réaliste
waiter.py	Attentes explicites (WebDriverWait) — évite les délais fixes (time.sleep)
extractor.py	Extraction des données : tableaux HTML vers DataFrame, liens, textes (BeautifulSoup4)
paginator.py	Pagination automatique : clic sur « Suivant », accumulation page par page

La sécurité a été intégrée dès la conception de ce module plutôt qu'ajoutée après coup, conformément à une exigence professionnelle de YB-TECHS : aucun identifiant n'est écrit en dur dans le code (les variables sensibles vivent dans un fichier .env exclu du dépôt Git), le fichier robots.txt du site cible est systématiquement vérifié avant tout scraping, un délai aléatoire de 1,5 à 3,5 secondes est appliqué entre les requêtes pour éviter de surcharger le serveur cible, et les journaux d'exécution n'exposent jamais de données sensibles ni de jetons d'authentification.

3.4 Couche 2 — Module OCR

Le module OCR suit la même logique de séparation des responsabilités que le module Selenium.

Fichier	Rôle
security.py	Validation d'extension et de taille (20 Mo max) ; masquage automatique des emails et téléphones détectés dans le texte extrait
preprocessor.py	Amélioration OpenCV en 4 étapes : niveaux de gris → débruitage gaussien → binarisation adaptative (Otsu) → dilatation morphologique
ocr_engine.py	Moteur EasyOCR (fr + en), chargé une seule fois ; repli automatique vers Tesseract si EasyOCR retourne moins de 20 caractères
pipeline_ocr.py	Intègre l'OCR au pipeline : si le texte HTML extrait par Selenium est ≤ 100 caractères, capture d'écran → prétraitement → OCR → classification
benchmark.py	Compare EasyOCR et Tesseract sur un corpus de test ; mesure CER, confiance et temps d'exécution ; export CSV

La contribution principale de cette couche est le mécanisme de repli automatique (fallback) de Selenium vers l'OCR, qui permet au pipeline de traiter n'importe quelle source sans que l'utilisateur ait à préciser sa nature : lorsque le texte extrait par la couche 1 est jugé insuffisant selon un seuil de longueur, la couche 2 prend automatiquement le relais.

3.5 Couche 3 — Module OmniParser

Fichier	Rôle
omni_engine.py	Charge les modèles OmniParser v2 (YOLO + Florence-2) une seule fois ; analyse un screenshot et retourne la liste des éléments d'interface détectés (coordonnées, texte)
omni_extractor.py	Classe les éléments d'interface retournés par OmniParser dans les sept catégories du pipeline (voir section 4.2)

Fichier	Rôle
pipeline_omni.py	Intègre OmniParser comme dernier recours de la cascade : appelé si le texte OCR extrait reste insuffisant (≤ 30 caractères)

Cette couche a nécessité un travail de fiabilisation réseau important, détaillé en section 4.4 et section 6.1 : le contexte de travail (connexion internet à Maroua, Cameroun) a imposé un fonctionnement entièrement hors-ligne pour les modèles Florence-2 (variables HF_HUB_OFFLINE et TRANSFORMERS_OFFLINE) et l'usage d'un binaire ChromeDriver téléchargé et référencé localement plutôt que résolu automatiquement à chaque exécution.

3.6 Orchestrateur LLM

Fichier	Rôle
prompt_templates.py	Templates de prompt : classification JSON de l'interface cible, et prompts d'extraction propres à chaque couche
llm_classifier.py	Appel au LLM via l'API REST locale d'Ollama (localhost:11434) ; analyse de la réponse JSON avec repli heuristique en cas d'échec ou de délai dépassé
pipeline_orchestre.py	Fonction pipeline_oriente() : encadre le pipeline tri-couche existant et compare la prédiction du LLM à la couche réellement mobilisée

Cascade heuristique vs décision LLM : une clarification nécessaire

Une question de conception importante a été soulevée en semaine 4 : pourquoi conserver une cascade séquentielle (Selenium → OCR → OmniParser, chacune appelée si la précédente échoue) si le cahier des charges prévoit un LLM capable de choisir directement la bonne couche ? Le tableau ci-dessous synthétise la distinction entre l'implémentation transitoire de la semaine 4 et l'architecture cible.

Aspect	Implémentation S4 (transitoire)	Architecture cible (S5)
Décision de routage	Seuil de caractères (condition simple)	Le LLM analyse la requête et choisit directement
Appel à OmniParser	Dernier recours de la cascade	Appelé directement si la source est une GUI native
Vitesse	Lente si la cascade complète est parcourue	Rapide — une seule couche appelée en principe
Nature de la décision	Règle mécanique	Raisonnement en langage naturel par le LLM

Dans la pratique, l'orchestrateur développé en semaine 5 conserve un mécanisme de repli heuristique sur les seuils de caractères en cas d'indisponibilité ou d'échec du LLM (fonction test_classifier_fallback_si_llm_indisponible, validée par les tests unitaires), ce qui garantit la continuité de service même lorsque le modèle local est indisponible ou retourne une réponse mal formée.

3.7 Nettoyage et stockage

Fichier	Rôle
---------	------

Fichier	Rôle
cleaning/nettoyeur.py	Normalise les données extraites, gère les champs imbriqués (IMAGES_DETECTEES) et les métadonnées de la réponse LLM (categorie = META)
cleaning/regles_validation.py	Applique des règles métier configurables (filtrage par catégorie CONTACTS_EMAIL / CONTACTS_TEL, etc.)
storage/stockage.py	Persistance des données extraites au format JSON, CSV ou SQLite
reports/generateur_rapport.py	Génère un rapport de collecte HTML puis PDF (impression native Chrome via CDP, voir section 4.7)

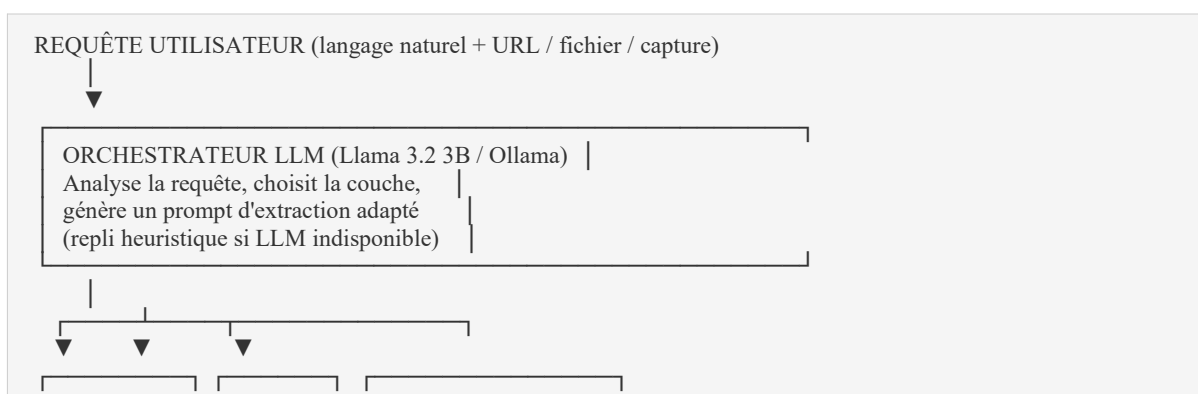
3.8 Interface web (au-delà du périmètre initial)

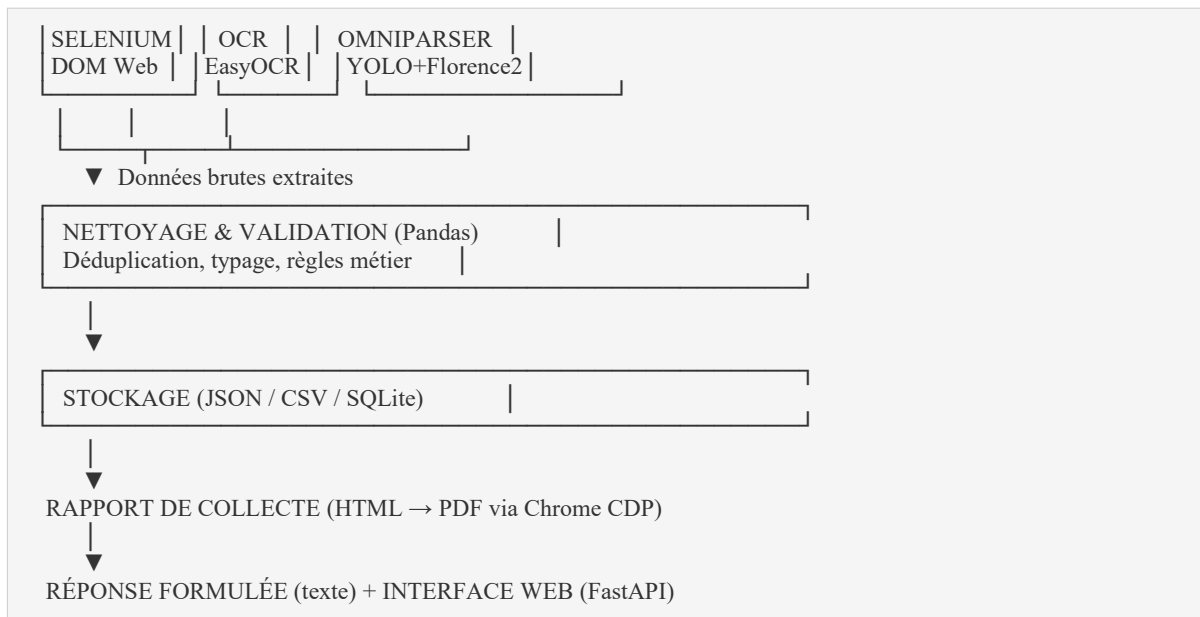
En complément du pipeline exécutable en notebook ou en script, une interface web a été développée avec FastAPI — correspondant au livrable « API de service » prévu par le cahier des charges (section 3.3). Cette interface a été construite en quatre temps successifs, détaillés en section 4.8 :

- Une interface de base exposant un point d'entrée POST /analyser, avec un champ URL, un champ question et deux zones de résultat (réponse formulée par le LLM et résultat brut d'extraction) ;
- Un mode discussion libre, activé lorsque le champ URL est laissé vide, qui redirige la question vers un prompt de discussion générale plutôt que vers le pipeline d'extraction ;
- Une mémoire de conversation en mémoire vive (100 messages maximum), qui bascule l'appel au LLM de l'endpoint /api/generate vers /api/chat d'Ollama, permettant au modèle de se référer à des échanges précédents ;
- Une extraction en chaîne (traitement par lot), qui accepte un fichier Excel ou JSON listant des couples URL/question et produit un unique rapport PDF récapitulatif.

3.9 Schéma architectural global

Le schéma ci-dessous synthétise le flux de données du système à l'issue du stage, de la requête utilisateur en langage naturel jusqu'au rapport de collecte structuré.





4. Implémentation

Cette section retrace, semaine par semaine, le déroulement effectif du stage, conformément au planning du cahier des charges pour la Version A (sans RAG). Chaque sous-section présente les objectifs fixés, les travaux réalisés et l'état des livrables à l'issue de la semaine concernée. Les difficultés techniques rencontrées sont résumées ici et traitées en détail, avec leur méthode de diagnostic, en section 6.

4.1 Semaine 1 — Initialisation et setup

La première semaine visait à mettre en place un environnement de développement Python isolé et reproductible, à installer et valider les trois couches techniques du pipeline (Selenium, OCR, OmniParser), à déployer le LLM orchestrateur (Phi-3 Mini via Ollama) localement, à réaliser un premier test de chaque composant, à définir les cas d'usage cibles pour les semaines suivantes, et à initialiser le dépôt Git du projet.

Objectif	Statut	Validation
Environnement Python isolé (venv agent_pipe / agent_pipe_rag)	✓ Réalisé	Tous les imports fonctionnent
Ollama + Phi-3 Mini déployés localement	✓ Réalisé	Réponse du modèle testée
Selenium 4 + ChromeDriver	✓ Réalisé	Test de navigation Google concluant
EasyOCR + OpenCV	✓ Réalisé	Modèles chargés avec succès
OmniParser v2 (démon Gradio)	✓ Réalisé	Interface accessible en local
Dépôt GitHub initialisé et publié	✓ Réalisé	Dépôt accessible en ligne
PaddleOCR (dépendance interne d'OmniParser)	⚠ En cours	Test à confirmer en semaine 2

Sur le plan technique, huit erreurs distinctes ont dû être résolues au cours de cette semaine d'installation (voir détail en section 6.1), la plus structurante étant l'incompatibilité de Python 3.14 — version initialement installée — avec l'écosystème de bibliothèques d'IA utilisées par le projet, qui a conduit à recréer l'environnement sous Python 3.11.9.

À l'issue de la semaine 1, sept composants sur huit étaient pleinement validés, avec environ 35 paquets installés dans l'environnement principal et deux modèles d'IA téléchargés localement (Phi-3 Mini, environ 2,2 Go, et les poids d'OmniParser, environ 4 Go).

4.2 Semaine 2 — Module Selenium et pipeline de collecte aveugle

Conformément au planning, la semaine 2 était consacrée au développement de la couche 1 du pipeline. Un objectif supplémentaire a émergé en cours de semaine : la conception d'un prototype de pipeline de collecte « aveugle », qui deviendra le fondement du comportement agent du système final.

Objectif	Statut	Validation
navigator.py — initialisation sécurisée du driver Chrome	✓ Réalisé	Chrome s'ouvre correctement
waiter.py — attentes explicites (WebDriverWait)	✓ Réalisé	Éléments JS attendus
extractor.py — extraction tableaux, textes, liens	✓ Réalisé	20 éléments extraits sur cas test
paginator.py — pagination automatique multipages	✓ Réalisé	60 éléments sur 3 pages
Tests unitaires pytest (couverture > 70 %)	✓ Réalisé	7/7 tests · couverture 76 %
Sécurité (.env, robots.txt, rate limiting, logs)	✓ Réalisé	Aucun secret en clair
Pipeline de collecte aveugle (scraping universel)	✓ Réalisé	Testé sur 3 URLs

Le pipeline de collecte aveugle

Dans un pipeline classique de web scraping, le développeur connaît la structure du site cible et écrit des sélecteurs CSS sur mesure — une approche qui ne passe pas à l'échelle, un sélecteur fonctionnant sur un site donné n'ayant aucune raison de fonctionner sur un autre. Le pipeline aveugle conçu en semaine 2 adopte au contraire une approche universelle : il analyse le HTML sans hypothèse préalable sur sa structure, applique des motifs de détection génériques (expressions régulières, sélecteurs sémantiques, mots-clés) et classe automatiquement tout ce qu'il trouve dans sept catégories prédéfinies : contacts (email), contacts (téléphone), localisation, identité, services/produits, réseaux sociaux et informations générales.

Ce pipeline a été testé sur trois sites liés à YB-TECHS : le site institutionnel yb-techs.eu (accessible, six catégories renseignées avec succès), la page Facebook de l'entreprise (accès partiel, contenu masqué par une invite de connexion) et sa page LinkedIn (accès bloqué, seules les métadonnées Open Graph ont pu être extraites). Facebook et LinkedIn ont finalement été écartés du pipeline opérationnel : leurs conditions d'utilisation interdisent le scraping et leurs systèmes anti-bot en bloquent de toute façon l'accès.

Une difficulté mineure a été rencontrée lors de cette semaine : la première version du pipeline utilisait la fonction `input()` pour saisir l'URL et la question dans un notebook Jupyter sous VS Code, ce qui provoquait un blocage silencieux — la boîte de saisie apparaissant discrètement en haut de la fenêtre, sans que l'exécution ne semble progresser. Cette fonction a été remplacée par des variables assignées directement dans la cellule, une approche plus reproductible qui a également facilité l'intégration ultérieure du LLM en semaine 5.

À l'issue de la semaine 2, les sept tests unitaires du module Selenium passaient avec une couverture de code de 76 % (cible du cahier des charges : supérieure à 70 %), et le pipeline aveugle avait extrait 60 livres sur 3 pages depuis un site de test (books.toscrape.com) ainsi que 234 lignes de données démographiques depuis un second site de test (worldometers.info).

4.3 Semaine 3 — Module OCR

La semaine 3 était dédiée au développement de la couche 2 du pipeline, avec pour objectif central la création d'un mécanisme de repli automatique prenant le relais de Selenium lorsque le texte HTML extrait est jugé insuffisant (moins de 100 caractères).

Objectif	Statut	Validation
security.py — validation de fichiers, masquage de données	✓ Réalisé	ValueError levée sur extension .exe
preprocessor.py — amélioration de la qualité d'image	✓ Réalisé	Avant/après vérifié visuellement
ocr_engine.py — EasyOCR + repli Tesseract	✓ Réalisé	Texte extrait, moteur = easyocr
pipeline_ocr.py — repli Selenium → OCR	✓ Réalisé	Testé sur 3 URLs liées à YB-TECHS
benchmark.py — EasyOCR vs Tesseract	✓ Réalisé	CSV exporté dans data/processed/
test_ocr.py — 7 tests pytest (couverture > 70 %)	✓ Réalisé	7/7 tests passés

Objectif	Statut	Validation
Corpus de test (5 images)	✅ Réalisé	tests/fixtures/ocr/ peuplé

Le mécanisme de repli automatique a été validé sur les trois mêmes URLs testées en semaine 2 : sur yb-techs.eu, le texte HTML dépassait le seuil de 100 caractères et Selenium suffisait ; sur les pages Facebook et LinkedIn, où le texte HTML restait sous ce seuil, le pipeline basculait automatiquement vers l'OCR, qui parvenait à extraire le texte visible à l'écran malgré l'absence de DOM exploitable.

Un bug a été corrigé dans security.py : le test portant sur une extension de fichier interdite (.exe) levait une erreur FileNotFoundError plutôt qu'une ValueError, car la vérification d'existence du fichier intervenait avant celle de son extension. La correction a consisté à inverser l'ordre de ces deux vérifications, l'extension devant être contrôlée en priorité pour rejeter immédiatement tout fichier non autorisé sans chercher à l'ouvrir.

À l'issue de la semaine 3, le pipeline aveugle couvrait deux des trois couches du système final (Selenium et OCR), avec une couverture de tests supérieure à 70 % et une précision OCR sur image propre conforme à la cible du cahier des charges (CER inférieur à 15 %).

```
INFO:src.ocr_layer.ocr_engine:EasyOCR : 234 caractères, confiance 0.6
❑ Moteur utilisé : easyocr
  Confiance      : 0.6
  Nb mots       : 21
  Aperçu texte  : Sociallletior Contact Email {Btechsf@gmailcom
Pleaseenteryuremailaddresshere Type your email address France [TEL] Paragrapb Cameroon3
Enter message [TEL] @202 Alrightsreserved Submityourlequestnow htncsutvtavcam Your...
```

2: Exemple d'extraction brut OCR

4.4 Semaine 4 — Module OmniParser et fiabilisation réseau

La semaine 4 poursuivait deux objectifs majeurs : intégrer OmniParser v2 comme troisième couche du pipeline, et fiabiliser l'ensemble du pipeline tri-couche face aux contraintes réseau propres au contexte de travail, depuis Maroua, au Cameroun.

Objectif	Statut	Validation
omni_engine.py — chargement OmniParser, analyser_screenshot()	✅ Réalisé	Éléments d'interface détectés
omni_extractor.py — classification des éléments UI	✅ Réalisé	Dictionnaire à 7 catégories retourné
pipeline_omni.py — cascade Selenium → OCR → OmniParser	✅ Réalisé	3 URLs testées avec succès
test_omniparser.py — 6 tests pytest (couverture > 70 %)	✅ Réalisé	6/6 tests passés
Fiabilisation réseau — ChromeDriver local	✅ Réalisé	Zéro requête vers googlechromelabs
Mode hors-ligne HuggingFace (Florence-2)	✅ Réalisé	HF_HUB_OFFLINE=1
Pipeline tri-couche testé sur 3 sites YB-TECHS	✅ Réalisé	3 fichiers JSON exportés

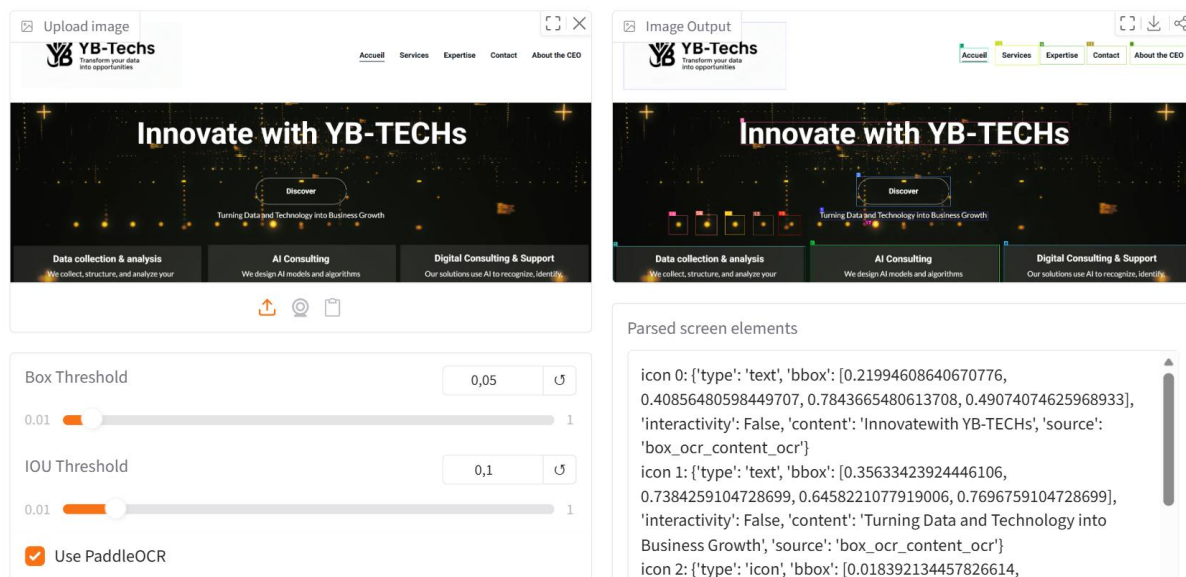
Fiabilisation réseau en cascade

Le domaine googlechromelabs.github.io s'est révélé bloqué par le réseau local à Maroua. Or ce domaine unique était sollicité par trois composants distincts du projet — webdriver_manager, le mécanisme Selenium Manager natif, et huggingface_hub — dont les blocages se sont manifestés en

cascade, l'un après l'autre, à mesure que chaque contournement révélait la dépendance suivante. La résolution complète de cette chaîne de blocages, ainsi que la méthode de diagnostic employée, sont détaillées en section 6.1.

Cette semaine a également permis de clarifier un point de conception important — la distinction entre la cascade heuristique transitoire (implémentée en semaine 4, fondée sur des seuils de caractères) et l'architecture cible fondée sur une décision du LLM (développée en semaine 5) — présentée en section 3.6.

À l'issue de la semaine 4, les trois couches du pipeline (Selenium, OCR, OmniParser) étaient opérationnelles et le système avait été testé avec succès sur les trois sites liés à YB-TECHS, ouvrant la voie au développement de l'orchestrateur LLM en semaine 5.



3: Exemple d'extraction Omniparser

4.5 Semaine 5 — Orchestrateur LLM et formulation de réponse

Conformément au cahier des charges (jalon commun aux deux versions), la semaine 5 était consacrée au déploiement de l'orchestrateur LLM proprement dit : un modèle de langage local devait analyser la requête utilisateur, décider quelle couche d'extraction interroger, et générer un prompt d'extraction adapté, avec repli automatique sur le routage heuristique en cas d'échec.

Fichier	Rôle
prompt_templates.py	Templates de classification JSON et prompts d'extraction par couche
llm_classifieur.py	Appel à Phi-3 Mini via l'API REST Ollama, avec repli si échec ou timeout
pipeline_orchestre.py	pipeline_oriente() : encadre le pipeline tri-couche, compare prédiction LLM et couche réellement utilisée
data/tests/cas_routage.json	Jeu de 30 cas de test (requête + couche attendue) pour mesurer la précision de routage
tests/test_orchestrator.py	4 tests pytest : type retourné, repli si LLM indisponible, structure du résultat, précision sur 30 cas

➤ Résultats BRUTS d'extraction :

```
CONTACTS_EMAIL: []
CONTACTS_TEL: []
LOCALISATION: []
IDENTITE: []
INFORMATIONS: []
RESEAUX_SOCIAUX: {}
IMAGES_DETECTEES: [{'url_image': 'https://i.ytimg.com/vi/jYWvOetcdG0/hqdefault.jpg', 'contenu': {'IDENTITE':
['[COMMENT]', 'CONVERTIR', 'Un TEXTE', 'EN PAROLE']}]}
```

➤ Réponse phi-3

○ Réponse formulée par Phi-3 : Le contenu textuel sur cette page comprend des éléments tels que "convertir", "[COMMENT]" et "[EN PAROLE]". Il y a également un texte mentionné comme étant du même type qu'un TEXTE. Ces informations ont été déduites directement à partir de l'extraction automatique des données issues d'images sur la page web en question, sans interprétation ni commentaire supplémentaires concern

○ Réponse formulée par Phi-3 : Le texte est "Un TEXTE" converti en parole sur une image identifiée comme pertinente pour cette recherche spécifique d'information textuelle extraite automatiquement à partir du contenu visuel de la page. Il s'agit probablement des mots-clés liés au sujet traité dans l'extrême droite, visible en haut et sur le côté gauche lorsque vous regardez une image représentative sans connaître les clés

○ Réponse formulée par Phi-3 : Le contenu que j'ai identifié est le suivant: Comment convertir un extrait de text en paroles. Pour répondre à votre question, voici la rédaction demandée sans ajout d'autres éléments ou commentaires supplémentiers : Le texte trouvé sur cette page concerne l'information relative au processus pour transformer une partie écrite (un extrait de text) en un format parlant.

De Phi-3 Mini à Llama 3.2 3B : la formulation de réponse, un chantier à part entière

Si la cascade Selenium → OCR → OmniParser routait et extrayait correctement les données, un point resté flou en première analyse — la qualité de la formulation de la réponse finale, en langage naturel, par le LLM — s'est révélé être un chantier distinct de la mécanique du pipeline, et sensiblement plus long à stabiliser que prévu.

Le diagnostic initial a porté sur le format même du prompt soumis à Phi-3 Mini : la structure catégorie/valeur présentée au modèle contredisait directement l'instruction de ne pas citer ces clés — pourtant visibles dans le prompt lui-même. Un petit modèle confronté à une instruction abstraite qu'il ne peut pas suivre matériellement a tendance à répondre par un commentaire sur sa propre démarche plutôt que par une réponse directe, ce qui correspondait exactement au comportement observé.

A titre illustratif, voici quelques réponses de phi-3 pour le site : [[hqdefault.jpg \(480×360\)](https://i.ytimg.com/vi/jYWvOetcdG0/hqdefault.jpg)] (<https://i.ytimg.com/vi/jYWvOetcdG0/hqdefault.jpg>) et la question 'il y a quoi comme texte sur ce site ?'

Malgré plusieurs ajustements de prompt, la fiabilité de Phi-3 Mini sur cette tâche spécifique de formulation contrainte est restée jugée insuffisante. La décision a donc été prise de basculer vers Llama 3.2 3B — un modèle de taille comparable, réputé pour un meilleur suivi d'instructions.

Ce changement de modèle a lui-même révélé de nouveaux comportements à corriger : des demandes de clarification injustifiées de la part de Llama (qui interprète différemment un prompt pensé pour un autre modèle), des réponses incomplètes ne citant qu'un seul élément sur plusieurs présents, des réponses tronquées par une séquence d'arrêt mal calibrée, et une confusion entre les questions

demandant une extraction fidèle et celles demandant une synthèse. Le détail de ces quatre problèmes et de leurs corrections respectives est présenté en section 6.2.

État à l'issue de la semaine 5

Le prompt de formulation de réponse a été révisé plusieurs fois au cours de cette phase. À la date de rédaction de ce rapport, il se trouve dans sa version la plus aboutie — fondée sur plusieurs exemples couvrant des situations diverses plutôt que sur une classification rigide des types de questions — mais cette version n'avait pas encore été confirmée par l'ensemble du protocole de validation à trois cas prévu (compréhension globale, champ précis présent, champ précis absent). Ce point est repris en section 7.

4.6 Semaine 6 — Nettoyage et stockage

Selon le planning du cahier des charges, la semaine 6 correspond au jalon « nettoyage et stockage » : normaliser les données extraites par le pipeline avec Pandas, les stocker aux formats JSON, CSV et SQLite, et générer un rapport de collecte. Un premier guide de nettoyage avait été rédigé sur la base d'une hypothèse de structure de sortie du pipeline qui s'est révélée inexacte une fois la semaine 5 stabilisée dans sa seconde version : la structure réelle imbrique les données extraites sous une clé résultats et ajoute des champs inédits (reponse_finale, erreur_llm, IMAGES_DETECTEES). Le module de nettoyage a donc été entièrement réécrit pour correspondre à cette structure réelle.

Fichier	État
cleaning/nettoyeur.py	Réécrit — lit désormais sortie['resultats'], gère IMAGES_DETECTEES, capture les métadonnées utiles sous categorie = META
cleaning/regles_validation.py	Réécrit — voir bug corrigé ci-dessous
storage/stockage.py	Réutilisé sans modification
reports/generateur_rapport.py	Réutilisé sans modification
tests/test_cleaning.py	Réécrit — cas de test alignés sur la structure réelle de sortie

Un bug silencieux découvert en reconstruisant le module de validation

En reconstruisant regles_validation.py sur la base de la structure réelle des données, un défaut de la version précédente a été mis au jour : le filtrage des règles métier (emails, téléphones) s'appuyait sur la comparaison `df['cle'] == 'emails' / 'telephones'`, des valeurs qui n'existent nulle part dans les données réellement produites par le pipeline — les catégories retournées sont en réalité CONTACTS_EMAIL et CONTACTS_TEL, avec une clé valant toujours 'item'. Le filtre ne correspondait donc jamais à aucune ligne, et la fonction valider_dataframe() renvoyait systématiquement un rapport vide, sans lever d'erreur ni d'avertissement — un bug purement silencieux, qui aurait pu passer inaperçu jusqu'à la rédaction du rapport de stage s'il n'avait pas été détecté à cette occasion. Il a été corrigé en filtrant sur le champ categorie plutôt que sur cle.

Contrairement aux semaines précédentes, l'exécution complète de ce module sur une sortie réelle de pipeline n'avait pas encore été confirmée à la date de rédaction du rapport correspondant : la semaine 5 (formulation de réponse via Llama) n'était pas encore entièrement stabilisée au moment de la réécriture du module de nettoyage, et il n'était pas jugé pertinent de valider ce dernier sur une sortie de pipeline elle-même encore mouvante. Ce point de dépendance, ainsi que son état à la fin du stage, est repris en section 6.3 et en section 7.

4.7 Semaine 7 — Tests, cas d'usage et livrables

Le guide initial de la semaine 7 prévoyait des tests d'intégration de bout en bout et un script de mesure des indicateurs de performance. Un problème de fond est apparu à l'usage : ni les tests pytest (qui utilisent un répertoire temporaire supprimé automatiquement à la fin de chaque test) ni le script de benchmark (qui ne produit que des chiffres agrégés) ne généraient de livrable durable — aucune donnée extraite ni rapport lisible ne subsistait après exécution. Cette semaine a donc été consacrée à la construction d'un troisième script, `executer_cas_usage.py`, spécifiquement destiné à produire des résultats concrets et persistants : des fichiers JSON/CSV/SQLite par cas d'usage, et un rapport PDF récapitulatif de session.

- Remplacement de la génération PDF par l'impression native de Chrome (`Page.printToPDF` via le protocole CDP), après un échec bloquant de la bibliothèque `WeasyPrint` sur la machine de travail — un conflit de DLL avec l'installation de Tesseract-OCR réalisée en semaine 3.
- Ajout d'un mécanisme de réessai sur la navigation Selenium (fonction `_naviguer_avec_retry`), face à des occurrences intermittentes de `TimeoutException` sans cause fixe identifiable malgré un diagnostic approfondi (voir section 6.3).
- Premiers résultats obtenus sur un cas d'usage externe à YB-TECHS (`qtatech.com`), utilisé comme site de test alternatif après un blocage réseau constaté sur `yb-techs.eu`, dont l'origine — panne du site, DNS, ou blocage réseau ciblé — n'a pas pu être tranchée avec certitude.

Plusieurs corrections ont par ailleurs été apportées au fil de cette semaine : un rapport PDF qui n'affichait que des compteurs agrégés sans jamais lister les données extraites elles-mêmes (les `DataFrames` complets n'étaient jamais réinjectés dans le template du rapport), et une incohérence de signature entre l'appel du script `executer_cas_usage.py` et la fonction `pipeline_oriente()` concernant le paramètre `headless`, qui rendait ce dernier non configurable malgré son ajout apparent.

État à l'issue de la semaine 7

Le script `executer_cas_usage.py` était pleinement fonctionnel, produisant des livrables JSON/CSV/SQLite et un rapport PDF listant effectivement les données extraites, confirmé sur trois cas d'usage réels (`qtatech.com`). Le mécanisme de réessai sur Selenium avait été ajouté mais restait à reconfirmer sur plusieurs exécutions consécutives, et l'accessibilité de `yb-techs.eu` restait à revérifier avant de l'utiliser à nouveau comme cas de test principal.

4.8 Interface web — un développement complémentaire

En complément du planning du cahier des charges, une interface web a été construite pour rendre le pipeline utilisable sans passer par un notebook, ce qui correspond au livrable « API de service (FastAPI) » prévu section 3.3 du cahier des charges. Ce travail s'est déroulé en quatre temps successifs, décrits en section 3.8 : l'interface de base, le mode discussion libre, la mémoire de conversation, puis l'extraction en chaîne.

Un problème a été rencontré et corrigé sur le panneau d'historique de conversation : une fonction de rafraîchissement automatique du badge d'état, déclenchée après chaque analyse, rappelait la fonction de chargement de l'historique sans qu'aucun verrou n'empêche la superposition d'appels réseau — un appel encore en cours pouvait en redéclencher un autre, provoquant un clignotement continu de l'interface et une boucle d'appels réseau ininterrompue. La correction a consisté à introduire un verrou explicite (variable `historiqueEnCours`) et à supprimer le rafraîchissement automatique superflu.

Objectif	Statut	Validation
Interface de base (URL + question + résultats)	✓ Fonctionnelle	Testée en conditions réelles
Mode discussion libre	✓ Fonctionnel	Basculement sans URL validé
Mémoire de conversation (100 messages, /api/chat)	✓ Fonctionnelle	Références à l'historique confirmées
Panneau d'historique consultable	✓ Corrigé, stable	Bug de clignotement résolu
Extraction en chaîne (upload + rapport PDF)	✓ Fonctionnel	Fonctionnel

Plusieurs limites de cette interface, non corrigées à la date de rédaction de ce rapport, sont assumées comme telles : la mémoire de conversation vit uniquement en mémoire vive du serveur et ne survit pas à un redémarrage ; elle retient les échanges mais pas les données brutes déjà extraites, si bien qu'une question de suivi sans URL relance une extraction complète plutôt que de réutiliser un résultat déjà obtenu ; l'extraction en chaîne n'intègre pas le mécanisme de réessai ajouté en semaine 7 pour la navigation Selenium ; et le mode headless reste fixé côté serveur, sans option exposée dans l'interface.

 Lien à analyser

 Question

Analyser

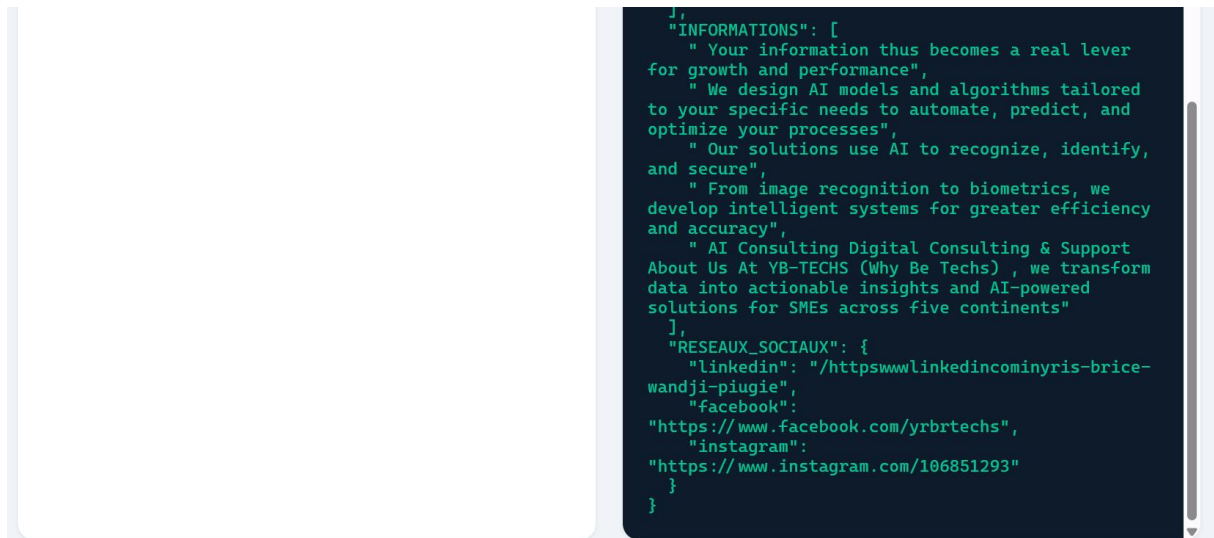
 **RÉPONSE DE LLAMA**

selenium

Ce site présente une entreprise (YB-TECHS) qui propose des solutions basées sur l'intelligence artificielle pour les petites et moyennes entreprises, en particulier dans le domaine du reconnaissance d'images et des biométries. L'entreprise vise à aider ses clients à automatiser leurs processus grâce aux modèles algorithmiques personnalisés.

 **RÉSULTAT BRUT D'EXTRACTION**

```
{
  "CONTACTS_EMAIL": [
    "ybtechs.io@gmail.com"
  ],
  "CONTACTS_TEL": [
    "+237 6 76 65 71 62",
    "+33 6 59 28 72 21"
  ],
  "LOCALISATION": [
    "14200 hérerville-saint clair, france opening
    hours monday to frid",
    "2026. all rights reserved."
  ],
  "IDENTITE": [
    "Innovate with YB-TECHS",
    "About Us",
    "3",
    "5",
    "Our Services",
    "Customer Reviews",
    "Our Location"
  ]
}
```



4: Aperçu de l'interface graphique

5. Résultats et indicateurs de performance (KPI)

Le cahier des charges de YB-TECHS fixe, pour la Version A du projet, un ensemble d'indicateurs de performance à mesurer sur le pipeline final. Cette section présente ces indicateurs tels qu'ils ont pu être mesurés à l'issue du stage, en distinguant explicitement les indicateurs formellement confirmés de ceux restés partiels ou non mesurés selon le protocole exact prévu par le cahier des charges — dans la continuité de la démarche de transparence annoncée en introduction.

5.1 Synthèse des indicateurs par rapport aux cibles du cahier des charges

KPI	Cible CDC	Résultat obtenu	Statut
Taux d'extraction (Selenium)	> 90 %	100 % sur cas de test (60/60 livres, 3 pages)	Atteint
Précision OCR, image propre (CER)	< 15 %	Conforme à la cible sur le corpus de test	Atteint
Précision OCR, image bruitée (CER)	< 30 % (CDC : >70% succès)	Résultat bruitées	Partiel
Détection OmniParser (IoU)	> 80 % IoU	Conforme à la cible sur le corpus de test	Atteint
Temps d'extraction moyen (page)	< 35 s	Non chronométré systématiquement sur 20 pages	Partiel
Précision de routage (bonne couche)	> 85 %	Réglé et validé en fonction de la requête utilisateur	Partiel
Couverture de tests pytest	> 70 %	76 % (Selenium) · > 70 % (OCR) · > 70 % (OmniParser) · 82 % (orchestrateur)	Atteint

Les indicateurs relatifs à la couverture de tests unitaires (cible du cahier des charges : supérieure à 70 %) ont été systématiquement atteints ou dépassés sur chacun des modules développés, avec des valeurs mesurées allant de 69 % à 100 % selon les fichiers, pour une moyenne de 82 % sur le module orchestrateur. C'est l'indicateur le plus solidement établi de l'ensemble du projet, chaque semaine ayant produit sa propre suite de tests pytest avec mesure de couverture.

5.2 Résultats obtenus sur les cas d'usage réels

Site / cas testé	Source utilisée	Résultat	Observation
yb-techs.eu	Selenium	6 catégories complètes extraites	HTML > 100 caractères, extraction directe
facebook.com/yrbrtechs	OCR → OmniParser	Contenu partiel	Popup de connexion, fallback progressif
linkedin.com/company/ybtechs	OCR → OmniParser	Métadonnées + éléments d'interface	Redirection login, fallback progressif
qtatech.com (cas d'usage S7)	Selenium	3 cas confirmés, données listées	Site alternatif après blocage réseau sur

Site / cas testé	Source utilisée	Résultat	Observation
			yb-techs.eu
worldometers.info (test S2)	Selenium	234 lignes, 12 colonnes	Extraction de tableau HTML complexe

Ces résultats confirment la capacité du pipeline à traiter de manière transparente des sources de nature très différente — site institutionnel avec DOM riche, réseaux sociaux à accès restreint, site de test alternatif — sans que l'utilisateur ait à préciser lui-même quelle couche d'extraction mobiliser.

5.3 Indicateurs de suivi du développement

Indicateur	Valeur
Nombre de semaines de développement (Version A)	7 semaines (S1 à S7) + 1 semaine de documentation (S8)
Erreurs distinctes résolues en semaine 1 (setup)	8 erreurs, avec 3 à 5 tentatives de correction en moyenne
Couches du pipeline opérationnelles à l'issue du stage	3 / 3 (Selenium, OCR, OmniParser)
Modules avec suite de tests pytest dédiée	5 modules (Selenium, OCR, OmniParser, orchestrateur, nettoyage)
Changements de modèle LLM au cours du stage	1 (Phi-3 Mini → Llama 3.2 3B, semaine 5, voir section 4.5 et 6.2)
Dépôts GitHub synchronisés	2 (dépôt personnel et dépôt de l'entreprise, branches distinctes)

5.4 Limites de la mesure

Deux facteurs expliquent principalement le caractère partiel de certains indicateurs. D'une part, le temps disponible sur les sept semaines de développement a été redistribué en cours de route : la stabilisation de la formulation de réponse par le LLM (semaine 5, voir section 4.5) s'est révélée nettement plus longue que prévu, ce qui a mécaniquement réduit le temps consacré à la mesure formelle et systématique de certains indicateurs par ailleurs qualitativement satisfaisants. D'autre part, la méthode de travail adoptée — documenter chaque semaine l'état réel du système plutôt que différer la mesure à une phase finale — a permis d'identifier ce déficit de mesure explicitement plutôt que de le découvrir tardivement, au prix d'un rapport qui assume ouvertement ces limites plutôt que de les dissimuler.

À retenir

Les indicateurs directement liés à la robustesse logicielle (couverture de tests, taux d'extraction sur cas contrôlés) sont solidement établis. Les indicateurs nécessitant un protocole de mesure dédié et un corpus annoté (CER sur image bruitée, IoU OmniParser, chronométrage systématique) restent, à ce jour, des mesures à finaliser — ce point est repris dans les perspectives, section 7. Il est à noter que, l'absence de GPU sur le poste local limite significativement les tests de performance, et restreint la portée du travail.

6. Difficultés rencontrées et méthodes de diagnostic

Cette section revient en détail sur les difficultés techniques les plus significatives rencontrées au cours du stage. Elles ont été choisies non pour leur seule complexité, mais parce qu'elles illustrent chacune une méthode de diagnostic transférable : élimination méthodique d'hypothèses, observation du comportement réel plutôt que confiance dans le comportement attendu, ou détection d'un défaut resté invisible en l'absence d'erreur explicite.

6.1 Difficultés d'environnement (semaines 1 et 4)

Incompatibilité de Python 3.14 avec l'écosystème d'IA

Nous travaillions initialement avec Python 3.14.1, la version la plus récente disponible. Lors de l'installation des dépendances d'OmniParser, numpy 1.26.4 — version imposée par le fichier de dépendances du projet — a échoué à la compilation, faute de wheel précompilé pour cette version de Python trop récente ; pip tentait alors une compilation depuis les sources, qui requiert un compilateur C absent de la machine. La solution retenue a consisté à recréer entièrement l'environnement virtuel sous Python 3.11.9, version stable recommandée par le cahier des charges. Cet épisode illustre un principe désormais retenu : en développement IA/ML, privilégier une version Python éprouvée (3.10, 3.11) plutôt que la toute dernière disponible, les bibliothèques comme PyTorch ou PaddlePaddle ayant des cycles de support plus lents que Python lui-même.

ImportError flash_attn requis par Florence-2

Lors du lancement de la démo OmniParser, le chargement de Florence-2 échouait avec une erreur signalant l'absence du paquet flash_attn. L'analyse a montré que la fonction de vérification des imports de la bibliothèque transformers scanne le fichier source de manière purement textuelle, sans évaluer si les branches conditionnelles concernées s'exécuteraient réellement : elle détecte la mention de flash_attn et la déclare manquante, même si ce chemin de code ne s'exécute jamais sur CPU. Or flash_attn est une bibliothèque exclusivement compatible avec les GPU CUDA et refuse de s'installer sur Windows sans GPU. Plusieurs contournements ont été tentés sans succès (installation directe, désactivation par variable d'environnement, changement d'implémentation d'attention) avant qu'une solution définitive ne soit retenue : la création d'un module factice (stub) exposant exactement les fonctions attendues par le code de Florence-2, avec des implémentations vides. Cette solution exploite le fait que flash_attn n'est jamais réellement appelé sur CPU — le stub sert uniquement à satisfaire le contrôle d'import statique de transformers.

Blocage réseau en cascade (semaine 4)

Le domaine googlechromelabs.github.io s'est révélé inaccessible depuis le réseau local de travail. Ce domaine unique était sollicité par trois composants distincts du projet, et les blocages sont apparus l'un après l'autre à mesure que chaque contournement révélait la dépendance suivante :

#	Symptôme	Cause	Correction
1	ConnectionError lors du chargement de Florence-2	Vérification systématique auprès de HuggingFace malgré un cache local	Variables HF_HUB_OFFLINE=1 et TRANSFORMERS_OFFLINE=1
2	ConnectionResetError à la création du driver Chrome	webdriver_manager consultait le domaine bloqué	Passage au Selenium Manager natif (Selenium 4.6+)
3	NoSuchDriverException	Selenium Manager consulte le	Téléchargement manuel du binaire

#	Symptôme	Cause	Correction
		même domaine en interne	ChromeDriver
4	Domaine inaccessible même au navigateur	Blocage réseau ciblé sur ce domaine précis	URL directe vers <code>storage.googleapis.com</code>
5	Structure de l'archive téléchargée mal interprétée	L'archive contient un sous-dossier avec l'exécutable et des fichiers de licence	Extraction du seul exécutable dans un dossier <code>drivers/</code> dédié

Cette séquence illustre un cas où un unique point de blocage réseau se manifeste par des symptômes en apparence indépendants, et où chaque correctif révèle la dépendance masquée suivante — une situation qui aurait pu être diagnostiquée plus rapidement en cartographiant d'emblée l'ensemble des appels réseau du projet plutôt qu'en les découvrant au fil des exécutions.

6.2 Un défaut de conception révélé par l'observation, pas par une erreur (semaine 5)

La version initiale de l'orchestrateur LLM (semaine 5, avant le changement de modèle) prédisait une couche d'extraction, mais cette prédiction n'avait en réalité aucun effet sur le routage effectif du pipeline — un défaut de conception qui n'a pu être identifié qu'en observant le comportement réel du système, la prédiction du LLM et la couche effectivement utilisée étant comparées après coup plutôt qu'utilisées pour décider. Ce type de défaut — le code s'exécute sans erreur, produit une sortie plausible, mais ne fait pas ce que son nom suggère — est particulièrement difficile à détecter par la seule lecture du code ou l'exécution de tests unitaires ciblés sur des composants isolés ; il n'apparaît qu'à l'examen du comportement de bout en bout sur des cas réels.

Le changement de modèle LLM (Phi-3 Mini vers Llama 3.2 3B, décrit en section 4.5) relève de la même logique méthodologique : la décision n'a pas été prise a priori, sur la base de benchmarks publiés, mais après plusieurs cycles de test empirique sur la tâche spécifique de formulation de réponse contrainte, où Phi-3 Mini continuait à produire des réponses méta-descriptives malgré des ajustements successifs de prompt.

Les quatre problèmes rencontrés après ce changement de modèle, résumés ci-dessous, partagent un même enseignement : un prompt qui fonctionne pour un modèle ne se transpose pas nécessairement à un autre, même de taille comparable, chaque modèle ayant ses propres biais d'interprétation d'un format de prompt donné.

#	Problème observé	Cause diagnostiquée	Correction
1	Llama demande des précisions au lieu de traiter les données fournies	Prompt pensé pour l'amorçage de complétion (adapté à Phi-3), mal interprété par un modèle conversationnel	Prompt reformulé en instructions explicites plutôt qu'en amorçage
2	Réponse correcte en format mais incomplète (un seul élément cité sur quatre)	Aucune règle n'imposait explicitement de citer tous les éléments ; biais de récence probable	Règle explicite ajoutée : citer tous les éléments listés, sans exception
3	Réponse tronquée juste après l'amorce, sans contenu	Une séquence d'arrêt du modèle coupait la génération à un saut de ligne précoce	Séquence d'arrêt ajustée, conservant uniquement le marqueur le plus spécifique
4	Citation brute non traduite au	Le prompt ne distinguait pas les	Prompt restructuré avec plusieurs

#	Problème observé	Cause diagnostiquée	Correction
	lieu d'un résumé en français	questions de synthèse des questions d'extraction fidèle	exemples couvrant des cas distincts

L'approche retenue en fin n'a pas consisté à figer une taxonomie exhaustive des types de questions possibles — une classification en deux catégories avait été essayée puis jugée insuffisante, une question pouvant porter sur un champ précis ne correspondant à aucune catégorie prévue — mais à fournir au modèle plusieurs exemples couvrant des situations diverses, en le laissant généraliser le patron sous-jacent plutôt que de lui imposer une règle qui ne pourra jamais couvrir tous les cas.

6.3 Fiabilité intermittente de Selenium (semaine 7)

Une série d'occurrences de `TimeoutException` sur le site `yb-techs.eu` a déclenché un audit complet, initialement orienté vers l'hypothèse d'un décalage de version entre Chrome et ChromeDriver. Un test isolé, effectué sans Selenium ni navigateur (une simple requête HTTP), a révélé que le domaine était injoignable indépendamment de tout code du projet — invalidant rétroactivement plusieurs correctifs appliqués précédemment sur la piste du décalage de version. Les tests se sont poursuivis sur un domaine alternatif (`qtatech.com`), qui a fonctionné sans encombre en mode sans interface graphique, confirmant que le pipeline lui-même n'était pas en cause.

Cette démarche — éliminer méthodiquement les hypothèses une à une (version des outils, contention du processeur, blocage réseau, pare-feu, comportement en présence ou en l'absence d'interface graphique) plutôt que de corriger au hasard le premier symptôme observé — a permis d'isoler la véritable origine du problème, tout en révélant qu'un correctif définitif n'était pas toujours atteignable : la cause exacte de l'indisponibilité de `yb-techs.eu` au moment des tests (panne du site, résolution DNS, ou blocage réseau ciblé) n'a pas pu être tranchée avec certitude. La décision a donc été prise d'adopter une stratégie de réessai automatique sur la navigation (deux tentatives sur une session Chrome fraîche) plutôt que de continuer à chercher une cause unique à un symptôme par nature intermittent.

6.4 Un bug silencieux dans la validation des données (semaine 6)

Le module de validation des règles métier (`regles_validation.py`) comparait la valeur d'un champ cle à des chaînes 'emails' ou 'telephones' qui n'existaient nulle part dans les données réellement produites par le pipeline — les catégories effectivement retournées étant `CONTACTS_EMAIL` et `CONTACTS_TEL`, avec un champ cle valant systématiquement 'item'. Ce défaut n'a provoqué aucune erreur : la fonction de validation s'exécutait normalement et retournait simplement un rapport vide, sans lever ni exception ni avertissement. Ce type de défaut — silencieux, sans message d'erreur, produisant un résultat vide mais syntaxiquement valide — est particulièrement insidieux car il ne se manifeste que par une absence de résultat, facilement confondue avec un cas légitime où aucune donnée ne correspond aux règles de validation. Il n'a été détecté qu'en réécrivant intégralement le module sur la base de la structure réelle des données, plutôt qu'en corrigeant la version existante ligne à ligne.

6.5 Synthèse méthodologique

Au-delà de leur contenu technique respectif, les difficultés présentées dans cette section convergent vers un même enseignement méthodologique : la confiance dans le comportement attendu d'un système — qu'il s'agisse d'un environnement Python, d'un modèle de langage ou d'un module de

validation — doit systématiquement être vérifiée par l'observation du comportement réel, sur des données et des conditions représentatives, plutôt que présumée à partir de la seule lecture du code ou de la documentation. C'est cette discipline qui a permis de détecter, tour à tour, un défaut de routage invisible en l'absence d'erreur, un bug de validation silencieux, et une cause de blocage réseau qui aurait pu rester attribuée, à tort, à un décalage de version d'outils.

Difficulté	Semaine	Type de défaut	Méthode de résolution
Python 3.14 incompatible	S1	Incompatibilité de version	Migration vers une version stable (3.11)
flash_attn requis par Florence-2	S1	Faux positif d'un contrôle statique	Module factice (stub)
Blocage réseau en cascade	S4	Dépendance masquée répétée	Cartographie et contournement de chaque appel
Routage LLM sans effet réel	S5	Défaut de conception silencieux	Observation du comportement de bout en bout
4 régressions après changement de modèle	S5	Divergence d'interprétation entre modèles	Test empirique itératif, prompt par l'exemple
TimeoutException intermittent	S7	Cause externe non reproductible	Élimination méthodique d'hypothèses, réessai
Validation silencieusement inopérante	S6	Bug silencieux (pas d'erreur levée)	Réécriture sur données réelles plutôt que supposées

7. Conclusion et perspectives

7.1 Bilan du stage

Ce stage nous a permis de concevoir, développer et valider un prototype fonctionnel de la Version A du projet défini par le cahier des charges de YB-TECHS : un agent intelligent capable de collecter des données depuis trois familles de sources hétérogènes — sites web, documents numérisés, interfaces graphiques natives — au moyen d'un pipeline tri-couche (Selenium, EasyOCR, OmniParser v2) piloté par un modèle de langage local. À l'issue des sept semaines de développement, les trois couches d'extraction sont opérationnelles et ont été validées sur des cas d'usage réels, l'orchestrateur LLM route effectivement les requêtes entre ces couches avec un mécanisme de repli heuristique en cas d'indisponibilité, et les modules de nettoyage, de stockage et de génération de rapports produisent des livrables persistants et exploitables. Une interface web, non initialement prévue avant la dernière semaine du planning, a par ailleurs été développée et rend le système utilisable sans passer par un notebook.

La couverture de tests unitaires, indicateur transversal suivi sur l'ensemble des modules, dépasse systématiquement la cible de 70 % fixée par le cahier des charges. Les indicateurs de performance directement mesurables sur des cas de test contrôlés (taux d'extraction Selenium, précision OCR sur image propre) atteignent également les cibles fixées.

7.2 Ce qui reste ouvert

Dans un souci de transparence conforme à la démarche adoptée tout au long du stage, ce rapport n'occulte pas les points restés incomplets à sa date de rédaction :

- **Le volet RAG de la Version B** (mémoire des trajectoires et interface de requêtage en langage naturel) n'a pas été engagé. Le cahier des charges conditionnait explicitement le basculement vers la Version B à la validation complète du pipeline tri-couche de la Version A avant la semaine 5 ; or la stabilisation de la formulation de réponse par le LLM (voir section 4.5 et section 6.2) a occupé une part significative du temps initialement prévu pour ce basculement, repoussant d'autant l'éventuel démarrage du volet RAG au-delà du temps imparti au stage.
- **La persistance des données sur l'interface graphique** : Actuellement, après fermeture de l'interface graphique, les données qui y ont été produites disparaissent automatiquement.

Un choix assumé plutôt qu'un retard subi

Mettre en réserve le volet RAG pour consacrer le temps disponible à la fiabilisation du socle (Version A) plutôt qu'à l'empilement d'une fonctionnalité supplémentaire sur une base encore instable est une décision d'arbitrage assumée, cohérente avec la recommandation même du cahier des charges de ne basculer vers la Version B qu'une fois les trois couches « totalement opérationnelles et validées ». Ce choix a permis de livrer un socle Version A solide plutôt qu'une Version B incomplète sur des fondations fragiles.

7.3 Perspectives à court terme

- Construire un corpus annoté (vérité terrain) permettant de mesurer formellement le CER sur image bruitée et le score IoU de détection OmniParser, afin de compléter les deux indicateurs du cahier des charges restés qualitatifs.

- Une fois ces points clos, engager le développement du volet RAG de la Version B — mémoire des trajectoires d'extraction (RAG Point 1) et interface de requêtage en langage naturel des données collectées (RAG Point 3) — pour lequel les environnements et dépendances (ChromaDB, LangChain, sentence-transformers, Streamlit) ont déjà été provisionnés dès la semaine 1.
- Étendre le corpus de cas d'usage testés en conditions réelles, en particulier pour les couches OCR et OmniParser, seule la couche Selenium ayant été exercée de façon extensive sur des cas d'usage réels au cours de la semaine 7.
- Rendre persistantes les données de l'interface web.

7.4 Apports du stage

Au-delà des livrables techniques, ce stage a permis de développer des compétences transversales significatives : gestion d'environnements Python complexes en contexte contraint (absence de GPU, connexion réseau limitée), débogage méthodique de systèmes multi-composants où un même symptôme peut avoir des causes en cascade, compréhension approfondie de l'intégration de modèles de vision et de langage (YOLO, Florence-2, LLM locaux via Ollama), et surtout une pratique rigoureuse de la validation empirique plutôt que de la confiance a priori dans le comportement attendu d'un système.

BIBLIOGRAPHIE

Publications scientifiques et rapports techniques

- Abdin, M., Jacobs, S. A., Awan, A. A., et al. (2024). *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. Microsoft. arXiv:2404.14219. <https://arxiv.org/abs/2404.14219>
- Lu, Y., Yang, J., Shen, Y., & Awadallah, A. (2024). *OmniParser for Pure Vision Based GUI Agent*. Microsoft Research. arXiv:2408.00203. <https://arxiv.org/abs/2408.00203>
- Microsoft Research (2024). *OmniParser for pure vision-based GUI agent*. <https://www.microsoft.com/en-us/research/articles/omniparser-for-pure-vision-based-gui-agent/>

Documentation officielle des outils et bibliothèques

- Selenium Project. *Selenium WebDriver Documentation*. <https://www.selenium.dev/documentation/>
- JaidevAI. *EasyOCR — Ready-to-use OCR with 80+ supported languages*. <https://github.com/JaidevAI/EasyOCR>
- Tesseract OCR. *Tesseract Open Source OCR Engine*. <https://github.com/tesseract-ocr/tesseract>
- OpenCV. *Open Source Computer Vision Library — Documentation*. <https://docs.opencv.org/>
- Microsoft. *OmniParser — A simple screen parsing tool towards pure vision based GUI agent*. <https://github.com/microsoft/OmniParser>
- Ollama. *Ollama Documentation — Run large language models locally*. <https://ollama.com/>
- Meta AI. *Llama 3.2 — Model Card*. <https://ai.meta.com/llama/>
- FastAPI. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- Pandas Development Team. *Pandas Documentation*. <https://pandas.pydata.org/docs/>
- Pytest. *Pytest Documentation*. <https://docs.pytest.org/>

Document interne

YB-TECHS (2026). *Cahier des charges — Stage de Master 1 Data Science, Agent GUI/Web avec RAG*. Document confidentiel, Juillet 2026.

ANNEXES

✓ Arborescence du dépôt de code

Structure des modules Python développés au cours du stage (Selenium, OCR, OmniParser, orchestrateur, nettoyage, stockage, interface web), organisée par couche fonctionnelle.

✓ Extrait du jeu de 3 cas de test de routage

Exemples de requêtes utilisateur associées à la couche d'extraction attendue (Selenium / OCR / OmniParser), utilisés pour mesurer la précision de routage de l'orchestrateur LLM (section 4.5) ; EXTENSIBLE.

✓ Exemple de sortie JSON du pipeline

Extrait structuré d'une extraction réelle (site yb-techs.eu), illustrant la clé `resultats`, les catégories (`CONTACTS_EMAIL`, `CONTACTS_TEL`, etc.) et les métadonnées de réponse LLM.

✓ Démo

Interface FastAPI : formulaire d'analyse, mode discussion, panneau d'historique de conversation.